

Ansible Mastery: A Comprehensive Guide for Interviews & Real-World Use

This document provides a detailed, yet concise, overview of key Ansible concepts, tailored for those seeking to gain interview-level proficiency and practical real-world skills. Each topic includes explanations and relevant examples.

1. Ansible

Definition: Ansible is an **open-source automation engine** that automates software provisioning, configuration management, and application deployment. It's designed to be simple, agentless, and highly effective for orchestrating complex IT workflows.

Real-time Example: Automating the entire setup of a new web server, from OS installation (via cloud APIs) to Nginx installation, configuration, and application deployment.

2. Open Source

Definition: "Open Source" means its **source code is publicly available** and can be freely used, modified, and distributed. Ansible benefits from a large community that contributes to its development, bug fixes, and feature enhancements.

Real-time Example: You can inspect Ansible's code on GitHub, contribute fixes, or build custom modules because it's open source. This also means no licensing costs for the core product.

3. Configuration Management

Definition: Configuration management is the process of **maintaining consistent and desired states of systems and software** across an infrastructure. It ensures that servers, applications, and network devices are configured correctly and identically, reducing manual errors and improving reliability.

Real-time Example: Ensuring all your production web servers have the same version of Apache, the same SSL certificate, and the same firewall rules applied.

4. Similar Tools in Market

Other prominent configuration management and automation tools:

- **Chef:** Agent-based, uses Ruby DSL.
- **Puppet:** Agent-based, uses its own Ruby-based DSL.
- **SaltStack:** Agent-based (minions), uses Python.
- **Terraform:** Infrastructure as Code (IaC) tool, focuses on provisioning, not strictly CM but complementary.

5. Developed by

Ansible was initially developed by **Michael DeHaan** (also the creator of Cobbler and Func) and later acquired by **Red Hat** in 2015.

6. SSH-22 Protocol

Role: Ansible primarily uses the **SSH (Secure Shell) protocol on port 22** to communicate with managed nodes (servers). This is its default and preferred connection method, requiring no special agents on the managed nodes.

Real-time Example: When Ansible runs a playbook, it establishes an SSH connection to each target server to execute commands, copy files, or install packages, just like you would manually from your terminal.

7. Agentless

Definition: Ansible is "agentless" meaning it **does not require any special software (agents) to be installed and running on the managed nodes** (the servers it controls). It simply uses standard SSH for communication and Python (or other scripting languages) that are typically already present on Linux/Unix systems.

Real-time Example: You can manage a newly provisioned EC2 instance immediately with Ansible as long as SSH is open and Python is installed, without needing to install a client daemon first.

8. Use Cases: Patching Activities

Ansible is excellent for patching activities due to its agentless nature and idempotent operations.

Real-time Example (Patching): You can use Ansible to:

- **Update OS packages:** `yum update -y` (Red Hat/CentOS) or `apt upgrade -y` (Debian/Ubuntu) across hundreds of servers.
- **Reboot servers:** After kernel updates.
- **Verify service status:** Ensure applications restart correctly after reboots or updates.
- **Rollback:** If an issue occurs, easily revert to previous configurations using version control for playbooks.

Other Use Cases:

- **Application Deployment:** Deploying WAR files to Tomcat, pushing code to Nginx/Apache.
- **Infrastructure Provisioning (via cloud modules):** Launching EC2 instances, configuring VPCs, setting up S3 buckets.
- **Orchestration:** Chaining together multiple steps across different systems (e.g., stopping load balancer, patching web servers, starting web servers, re-enabling load balancer).
- **Security Automation:** Enforcing firewall rules, managing user accounts, deploying security agents.

9. Most Used Ad-hoc Commands

Ad-hoc commands are single-line commands used for quick tasks, not stored in playbooks.

- **ansible all -m ping:** Checks connectivity to all hosts defined in your inventory.
 - **Explanation:** Uses the `ping` module to verify if Ansible can reach and authenticate with the target hosts.
 - **Real-time Example:** Quickly verify reachability after a network change or before running a major deployment.

- **ansible webservers -a "systemctl restart nginx":** Restarts Nginx service on hosts in the webservers group.
 - **Explanation:** Uses the command module (default for -a) to execute a raw shell command.
 - **Real-time Example:** Restarting a service on a group of servers after a manual config change.
- **ansible dbservers -m yum -a "name=mysql-server state=present":** Installs mysql-server on dbservers group.
 - **Explanation:** Uses the yum module to manage packages. state=present ensures it's installed.
 - **Real-time Example:** Quickly installing a package on multiple machines without writing a playbook.
- **ansible all -m file -a "path=/tmp/testfile state=absent":** Deletes /tmp/testfile on all hosts.
 - **Explanation:** Uses the file module to manage files and directories. state=absent ensures removal.
 - **Real-time Example:** Cleaning up temporary files across your infrastructure.

10. What is Hosts/Inventory?

Definition: The **inventory** (or hosts file) is a file that contains a list of the servers (managed nodes) that Ansible manages. It can group hosts and define variables specific to hosts or groups.

Real-time Example:

```
# inventory.ini
[webservers]
web1.example.com
web2.example.com

[dbservers]
db1.example.com ansible_port=2222

[development:children]
webservers
dbservers
```

```
[all:vars]
ansible_user=ec2-user
```

In this example, webserver and dbserver are groups, development is a group composed of other groups, and all:vars defines variables for all hosts. ansible_port overrides the default SSH port for db1.example.com.

Possible Ways We Can Configure It:

1. **INI Format (default):** Simple, human-readable key-value pairs and groups.
2. **YAML Format:** More structured, preferred for complex inventories with nested variables.
3. **Dynamic Inventory:** Scripts or plugins that pull inventory information from cloud providers (AWS, Azure, GCP), CMDBs (ServiceNow), or virtualization platforms (VMware, OpenStack). This is crucial for dynamic environments.

11. Plugins/Modules

Definition:

- **Modules:** Are reusable, standalone scripts that Ansible executes on managed nodes. They perform specific tasks, like installing packages, managing services, copying files, or interacting with APIs. Ansible ships with thousands of modules.
- **Plugins:** Extend Ansible's core functionality. Examples include:
 - **Connection Plugins:** How Ansible communicates (e.g., ssh, docker, local).
 - **Inventory Plugins:** How Ansible gathers inventory (e.g., aws_ec2).
 - **Callback Plugins:** What happens during a playbook run (e.g., logging, reporting).
 - **Lookup Plugins:** Fetch data from external sources (e.g., file, env, password).

Real-time Example:

- **Module:** apt (to install packages on Debian/Ubuntu), copy (to copy files), service (to manage services like Nginx).

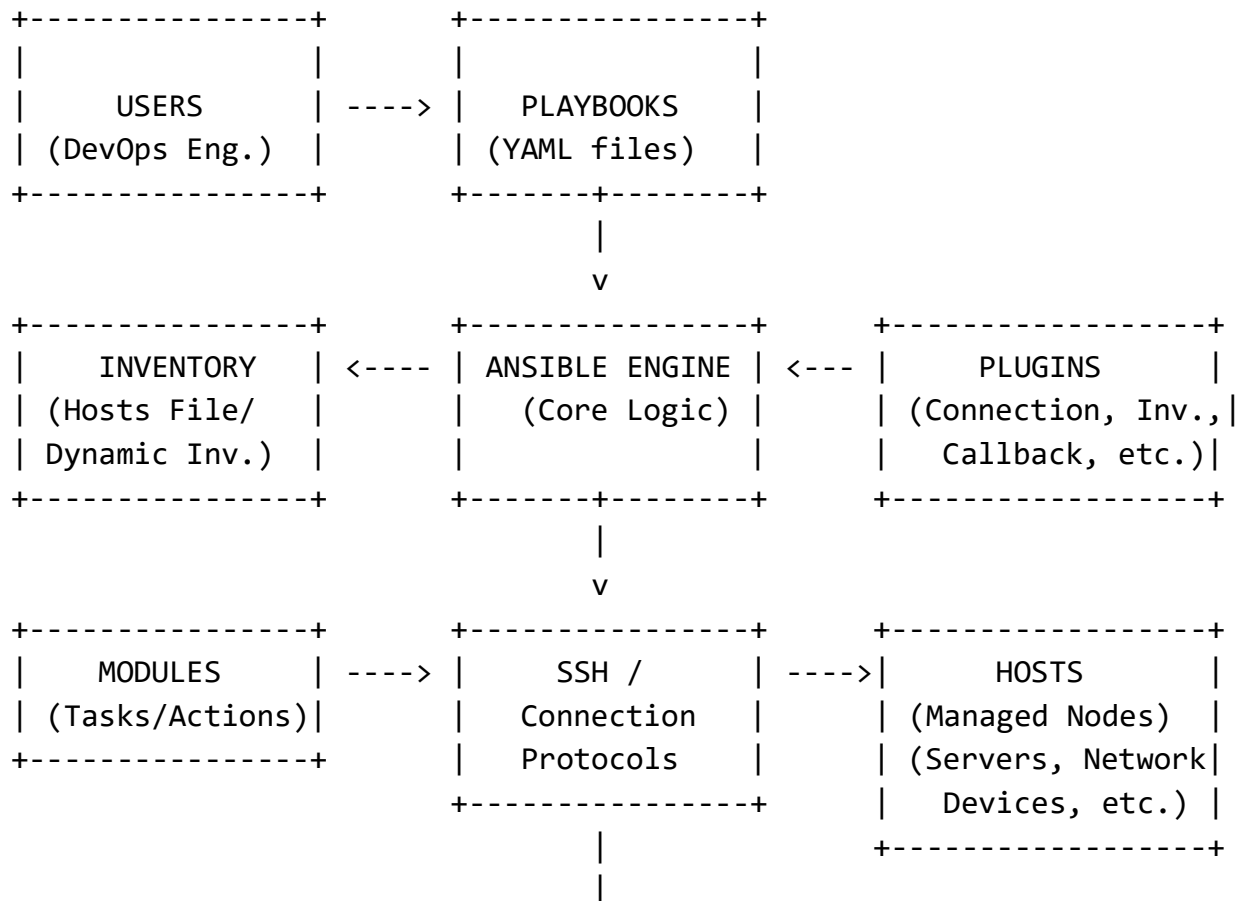
- **Plugin:** Using the aws_ec2 inventory plugin to automatically discover EC2 instances in your AWS account.

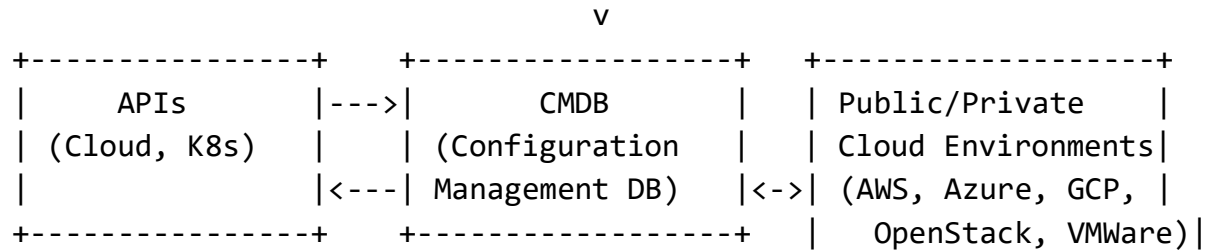
12. Ansible Controller and Workers

- **Ansible Controller (Control Node):** This is the machine where Ansible is installed, and from where playbooks are executed. It's the "brain" of your Ansible automation.
- **Managed Nodes (Workers/Targets):** These are the servers or devices that Ansible manages. They don't need any special software installed, just SSH access and typically Python (for Linux/Unix).

Real-time Example: Your laptop could be the Ansible Controller, managing a fleet of EC2 instances (Managed Nodes) in AWS.

13. Ansible Architecture Explanation (Block Diagram Type)





Explanation:

- **Users:** DevOps Engineers, System Admins, or Developers who write and execute Ansible Playbooks from the Control Node.
- **Playbooks:** YAML files that define a set of automation tasks, written by users. They are the core of Ansible automation.
- **Ansible Engine (Core Logic):** The installed Ansible software on the control node. It parses playbooks, inventories, and orchestrates the execution.
- **Inventory:** The list of managed hosts, categorized into groups, with associated variables.
- **Modules:** Individual units of code that perform specific actions on managed nodes (e.g., apt, yum, service, copy).
- **Plugins:** Extend Ansible's core functionality (e.g., how it connects, how it gathers facts, how it transforms data).
- **SSH / Connection Protocols:** Ansible's primary communication method with managed nodes (SSH for Linux, WinRM for Windows).
- **Hosts (Managed Nodes):** The target machines or devices that Ansible manages.
- **CMDB (Configuration Management Database):** External systems where inventory data or configuration details might reside. Ansible can integrate with CMDBs via dynamic inventory or lookups.
- **APIs:** Ansible can interact with various APIs (e.g., Cloud APIs for provisioning, Kubernetes APIs for deploying containers) using specific modules.
- **Public/Private Cloud Environments:** Where your managed hosts typically reside (e.g., AWS EC2 instances, Azure VMs, on-prem VMware VMs).

14. Ansible SSH Authentication Full Process (Professional)

1. Key Pair Management:

- a. **User Generates Keys:** The Ansible user generates an SSH key pair (private and public keys) on the Control Node (e.g., `ssh-keygen`).
- b. **Public Key Distribution:** The public key (`~/.ssh/id_rsa.pub` or similar) is securely copied to the `authorized_keys` file of the target user on each Managed Node (e.g., `~/.ssh/authorized_keys` for `ec2-user`).
 - i. **Real-time Example:** For cloud instances (AWS EC2), you inject the public key into the instance at launch time. For on-prem servers, you might use a tool like `ssh-copy-id` or a bootstrap script.

2. Ansible Configuration:

- a. **ansible_user:** In the inventory, `ansible_user` specifies the remote user (e.g., `ec2-user`, `ubuntu`, `root`) on the managed nodes that Ansible will SSH into.
- b. **ansible_private_key_file:** In the `ansible.cfg` or inventory, `ansible_private_key_file` points to the private key on the Control Node (`~/.ssh/id_rsa` or a custom key).
- c. **ansible_ssh_common_args / ProxyJump:** For multi-hop SSH (bastion hosts), you configure `ProxyJump` in `~/.ssh/config` or `ansible_ssh_common_args` in inventory to tell Ansible how to jump through an intermediary server.

3. Connection Establishment:

- a. When a playbook runs, Ansible's SSH connection plugin attempts to connect to each managed node using the specified `ansible_user` and the configured private key.
- b. The Managed Node's SSH daemon receives the connection request and checks the provided public key against the `authorized_keys` file for the `ansible_user`.

4. Authentication and Session:

- a. If the public key matches an entry in `authorized_keys`, the authentication is successful.
- b. An encrypted SSH session is established. Ansible then uses this secure channel to execute modules (which are temporary Python scripts) on the managed node.

5. Privilege Escalation (Optional):

- a. If tasks require root privileges, `become: yes` (with `become_method: sudo` and `become_user: root`) is used in the playbook. Ansible sends the `sudo` command (or configured alternative) through the established SSH session to gain necessary permissions for the task.

Security Best Practice: Always use SSH key-based authentication (not passwords) and configure appropriate file permissions for your private keys (`chmod 400`).

15. Real-time Modules and Their Uses

Ansible has thousands of modules. Here are some critical ones for real-time operations:

- **ansible.builtin.shell / ansible.builtin.command:** Executes arbitrary shell commands or simple commands.
 - **Use:** When a specific Ansible module doesn't exist for a niche task, or for piping/redirection that `command` can't do.
 - **Example:** `shell: df -h | grep '/dev/xvda1'` (Checks disk space).
- **ansible.builtin.apt / ansible.builtin.yum / ansible.builtin.dnf:** Manages packages on Debian/Ubuntu, Red Hat/CentOS, and Fedora/RHEL8+.
 - **Use:** Installing, updating, or removing software packages.
 - **Example:** `yum: name=httpd state=latest` (Ensures Apache is at the latest version).
- **ansible.builtin.service:** Manages services (start, stop, restart, enable at boot).
 - **Use:** Controlling application or system services.
 - **Example:** `service: name=nginx state=restarted enabled=yes` (Restarts Nginx and ensures it starts on boot).
- **ansible.builtin.copy:** Copies files from the control node to managed nodes.
 - **Use:** Deploying configuration files, scripts, or application binaries.
 - **Example:** `copy: src=/local/path/nginx.conf dest=/etc/nginx/nginx.conf owner=root group=root mode='0644'`
- **ansible.builtin.template:** Renders Jinja2 templates on the control node and copies the result to managed nodes.
 - **Use:** Dynamically generating configuration files based on variables. **Crucial for real-time config management.**
 - **Example:** `template: src=templates/my_app.conf.j2 dest=/etc/nginx/conf.d/my_app.conf`

- **ansible.builtin.file:** Manages files, directories, and symlinks (create, delete, change permissions).
 - **Use:** Ensuring directory structures, managing permissions.
 - **Example:** `file: path=/var/log/my_app state=directory mode='0755'`
- **ansible.builtin.lineinfile / ansible.builtin.blockinfile:** Inserts, replaces, or deletes lines/blocks in files.
 - **Use:** Making small, precise changes to configuration files without replacing the whole file.
 - **Example:** `lineinfile: path=/etc/ssh/sshd_config regexp='^#?Port' line='Port 2222' (Sets SSH port to 2222).`
- **ansible.builtin.user / ansible.builtin.group:** Manages users and groups.
 - **Use:** User provisioning, access control.
 - **Example:** `user: name=deployuser state=present home=/home/deployuser shell=/bin/bash`
- **community.general.ini_file / community.general.xml / community.general.json_patch:** Modules for specific file formats.
 - **Use:** Parsing and modifying structured configuration files.
- **Cloud Modules (e.g., community.aws.ec2_instance, community.aws.s3_bucket):** Interact with cloud provider APIs.
 - **Use:** Provisioning and managing cloud infrastructure.
 - **Example:** `community.aws.ec2_instance: name=webserver state=present instance_type=t2.micro image_id=ami-0abcdef12345 key_name=my-key`

16. when Condition in Playbooks

Definition: The when clause allows you to **conditionally execute a task** based on the result of a variable, a registered output, or a fact.

Real-time Example:

```
- name: Install Apache on Ubuntu servers
  ansible.builtin.apt:
    name: apache2
    state: present
  when: ansible_os_family == "Debian"
```

```
- name: Restart Nginx if config changed
  ansible.builtin.service:
    name: nginx
    state: restarted
  when: nginx_config_status.changed is defined and
        nginx_config_status.changed
```

- The first task only runs if the `ansible_os_family` fact (gathered automatically by Ansible) is Debian.
- The second task restarts Nginx only if a previous task that modified Nginx configuration reported a change (using `register`).

17. What is Playbook?

Definition: A playbook is a **YAML file that defines a set of automation tasks** to be executed on a specific group of hosts. Playbooks are the foundation of Ansible's configuration management and orchestration capabilities. They are declarative, meaning you describe the *desired state*, and Ansible figures out how to get there.

Real-time Example: A playbook can define all the steps to set up a new production server: install OS updates, install a web server, deploy an application, configure firewall, and start services.

18. Basic Playbook Structure

```
--- # YAML indicator, optional but good practice
- name: Configure Web Servers # Name of the playbook
  hosts: webservers           # Target group from inventory
  become: yes                 # Run tasks with elevated privileges
                              (e.g., sudo)

  tasks:                      # List of tasks to execute
    - name: Ensure Apache is installed
      ansible.builtin.yum:    # Module to use
        name: httpd
        state: latest        # Desired state (latest version)
```

- name: Start Apache service
 ansible.builtin.service:
 name: httpd
 state: started
 enabled: yes # Ensure it starts on boot

- name: Copy custom index.html
 ansible.builtin.copy:
 src: files/index.html # Local path on control node
 dest: /var/www/html/index.html # Remote path on managed node
 mode: '0644'

19. Ansible Playbook Common Directives/Keywords with Explanation

These are common top-level keywords within a playbook or task:

- **name:** (String) A descriptive name for the playbook, play, or task. Crucial for readability and debugging.
 - **Example:** - name: Install Nginx
- **hosts:** (String/List) Specifies which hosts or groups from the inventory this play will run on.
 - **Example:** hosts: webserver or hosts: all
- **become:** (Boolean) Whether to use privilege escalation (e.g., sudo) for tasks within this play. Default is no.
 - **Example:** become: yes
- **become_user:** (String) The user to become (switch to) when become: yes. Default is root.
 - **Example:** become_user: youradmin
- **tasks:** (List of dictionaries) The main sequence of actions to be executed on the managed hosts. Each item is a task definition.
 - **Example:** tasks: - name: ...
- **vars:** (Dictionary) Defines variables specific to this play or file.
 - **Example:** vars: { app_port: 8080 }

- **handlers:** (List of dictionaries) Defines handlers, which are tasks that only run when explicitly notified by another task.
 - **Example:** handlers: - name: restart_nginx
- **tags:** (String/List) Associates tags with plays or tasks for selective execution.
 - **Example:** tags: ['web', 'configure']
- **gather_facts:** (Boolean) Whether Ansible should collect facts (system information) about the managed hosts at the beginning of the play. Default is yes.
 - **Example:** gather_facts: no (useful for very fast runs or when facts aren't needed).
- **pre_tasks / post_tasks:** (List of dictionaries) Tasks to run before or after the main tasks block in a play, useful for setup/teardown.
- **roles:** (List) Specifies which roles to include in a play.
 - **Example:** roles: - webserver_role
- **remote_user:** (String) Specifies the remote user to connect as for this play (overrides ansible_user from inventory).

20. become: yes

Definition: become: yes is an Ansible directive that tells Ansible to **use privilege escalation** (e.g., sudo, su) to execute tasks as a different user, typically root.

Real-time Example: Installing a package or restarting a system service requires root privileges.

```
- name: Install Nginx package
  ansible.builtin.apt:
    name: nginx
    state: present
  become: yes # This task will run with sudo privileges
```

21. notify and handlers and How to Execute

notify:

- **Definition:** A notify statement in a task is used to **trigger one or more handlers** if that task makes a change on the managed node. It acts as a listener.

- **Real-time Example:** If a task changes a configuration file, you'd notify a handler to restart the associated service.

handlers:

- **Definition:** handlers are special tasks that are **only executed when explicitly notified** by a notify statement from another task. They run at the very end of a play, after all regular tasks have completed, and only run once even if notified multiple times by different tasks. This makes them idempotent for changes.
- **Real-time Example:** Restarting a service after its configuration file has been updated.

How to Execute:

1. Define a handler in the handlers section of your playbook, with a unique name.
2. In a task, add a notify statement referencing the handler's name.

Example:

```
---
- name: Deploy and Configure Web Server
  hosts: webservers
  become: yes

  tasks:
    - name: Copy Nginx configuration file
      ansible.builtin.template:
        src: templates/nginx.conf.j2
        dest: /etc/nginx/nginx.conf
        owner: root
        group: root
        mode: '0644'
        notify: restart nginx # This triggers the 'restart nginx'
                             handler IF the file actually changes

  handlers:
    - name: restart nginx
      ansible.builtin.service:
        name: nginx
        state: restarted
```

In this example, Nginx will *only* be restarted if the `nginx.conf` file was modified by the template task. If the template content hasn't changed, the template task reports "OK" (not "CHANGED"), and the handler is not notified.

22. register and debug

register:

- **Definition:** register is a task keyword that **captures the output (standard output, standard error, return code, changed status) of a task** and stores it in a variable on the Ansible Control Node.

- **Real-time Example:**

```
- name: Check Nginx service status
  ansible.builtin.command: systemctl status nginx
  register: nginx_status_output # Store the full output of the command
                                # in this variable
  ignore_errors: yes           # Allow the playbook to continue even
                                # if nginx is not running

- name: Print Nginx status
  ansible.builtin.debug:
    var: nginx_status_output # Print the entire registered variable
```

This allows you to inspect the output of a command or module for debugging, logging, or for use in subsequent when conditions.

debug:

- **Definition:** The debug module is used to **print variables or messages** during playbook execution. It's invaluable for troubleshooting and understanding playbook flow.
- **Real-time Example:**

- name: Show the value of a variable
`ansible.builtin.debug:`
`msg: "The application port is {{ app_port }}"` # Print a custom message with a variable
- name: Show the registered output of a previous task
`ansible.builtin.debug:`
`var: nginx_status_output.stdout_lines` # Only print the standard output lines of the registered variable
```var:`` prints the entire variable in YAML format, while ``msg:`` allows for custom formatted strings.

## 23. Ansible Vault

**Definition:** Ansible Vault is a feature that allows you to **encrypt sensitive data** (like passwords, API keys, SSH private keys) within your Ansible projects. This encrypted data can then be safely stored in version control (e.g., Git) alongside your playbooks, without exposing secrets.

### Real-time Use Cases:

- Storing database passwords in `group_vars/all/vault.yml`.
- Encrypting cloud API access keys.
- Securing SSH private keys for specific deployments.

### Ansible Vault Commands and Uses:

- **ansible-vault create <filename>:** Creates a new encrypted file.
  - **Use:** To create a new file specifically for secrets.
  - **Example:** `ansible-vault create group_vars/all/secrets.yml`  
 (Prompts for a new password).
- **ansible-vault edit <filename>:** Edits an existing encrypted file.
  - **Use:** To modify encrypted variables.
  - **Example:** `ansible-vault edit group_vars/all/secrets.yml`  
 (Prompts for vault password to decrypt, opens in editor, re-encrypts on save).



- **ansible-vault view <filename>:** Views an encrypted file without editing.
  - **Use:** To quickly check contents of a vault file.
  - **Example:** `ansible-vault view group_vars/all/secrets.yml`
- **ansible-vault encrypt <filename>:** Encrypts an existing unencrypted file.
  - **Use:** To encrypt a file that was previously plaintext.
  - **Example:** `ansible-vault encrypt my_database_passwords.yml`
- **ansible-vault decrypt <filename>:** Decrypts an existing encrypted file back to plaintext.
  - **Use:** For temporary decryption or if you no longer need it encrypted (use with caution!).
  - **Example:** `ansible-vault decrypt my_database_passwords.yml`
- **ansible-vault rekey <filename>:** Changes the encryption password of an encrypted file.
  - **Use:** For password rotation or if a vault password has been compromised.
  - **Example:** `ansible-vault rekey group_vars/all/secrets.yml`
- **Running Playbooks with Vault:**
  - **ansible-playbook my\_playbook.yml --ask-vault-pass:** Prompts for the vault password.
  - **ansible-playbook my\_playbook.yml --vault-password-file ~/.vault\_pass.txt:** Reads password from a file (ensure this file is secure and not in Git!).
  - **ansible-playbook my\_playbook.yml --vault-id production@prompt --vault-id dev@/path/to/dev\_vault\_pass:** For multiple vaults.

## 24. Variables with Explanation and Examples

Variables allow you to store values and use them in playbooks, making them more flexible and reusable. Ansible has a strict variable precedence.

- **Inline Variables (Task-level):** Defined directly within a task. Lowest precedence.
    - **Explanation:** Used for small, localized values that don't need to be shared.
    - **Example:**
- ```
- name: Install specific package
  ansible.builtin.yum:
    name: "{{ package_name }}"
    state: present
```

```
vars:
  package_name: httpd # Defined right here
```

- **vars File (Play/File-level):** Defined in a vars section within a playbook, or in a separate .yaml file imported via vars_files.
 - **Explanation:** Good for variables specific to a single playbook or a set of related plays.
 - **Example (in playbook):**

```
- name: Configure Web Application
  hosts: webservers
  vars:
    app_version: 1.2.3
    app_port: 80
```

```
tasks:
  - name: Deploy application version {{ app_version }}
    ansible.builtin.copy:
      src: "files/app-{{ app_version }}.tar.gz"
      dest: "/opt/app/app.tar.gz"
```

- **CLI Variables (-e or --extra-vars):** Passed directly on the command line. Highest precedence.
 - **Explanation:** Used for ad-hoc overrides or sensitive data that shouldn't be stored in files (though vault is better for secrets).
 - **Example:** ansible-playbook my_app.yaml -e "app_version=1.3.0 env=production"
- **Prompt Variables (vars_prompt):** Ansible prompts the user for a value during playbook execution.
 - **Explanation:** Useful for interactive input, especially for values that change frequently or are environment-specific during manual runs.
 - **Example:**

```
- name: Deploy Application
  hosts: webservers
```

```
vars_prompt:
  - name: "deploy_env"
    prompt: "Enter deployment environment (dev/stage/prod)"
    private: no # Show input
  - name: "db_password"
    prompt: "Enter database password"
    private: yes # Hide input
tasks:
  - name: Configure for {{ deploy_env }} environment
    ansible.builtin.template:
      src: templates/config.ini.j2
      dest: /etc/app/config.ini
```

- **Inventory Variables:** Defined within the inventory file itself, either for specific hosts or groups. High precedence.
 - **Explanation:** Ideal for defining host-specific or group-specific configurations.
 - **Example (in inventory.ini):**

```
[webservers]
web1.example.com app_port=80
web2.example.com app_port=443
```

```
[dbservers]
db1.example.com
```

```
[all:vars]
ansible_user=ubuntu
```

In this case, app_port would be 80 for web1 and 443 for web2.

25. Loops (loop)

Definition: Loops allow you to **repeat a task multiple times** with different input values, avoiding redundant task definitions.

Real-time Example: Installing multiple packages, creating multiple users, or managing multiple files.

- name: Install common packages
 ansible.builtin.yum:
 name: "{{ item }}"
 state: present
 loop:
 - vim
 - git
 - htop

- name: Create multiple users
 ansible.builtin.user:
 name: "{{ item.name }}"
 shell: "{{ item.shell | default('/bin/bash') }}"
 state: present
 loop:
 - { name: 'john', shell: '/bin/bash' }
 - { name: 'jane', shell: '/bin/sh' }
 - { name: 'bob' } # Uses default shell

- The first example installs vim, git, and htop sequentially.
- The second example creates three users, demonstrating looping over a list of dictionaries.

26. Tags in Ansible Command and Its Uses

Definition: tags are labels that can be applied to plays, tasks, or even roles within a playbook. They allow you to **selectively run or skip specific parts** of a playbook based on these labels.

Real-time Use Cases:

- Running only the "db" configuration tasks during a database upgrade.
- Skipping time-consuming "package_install" tasks during development.
- Executing only "rollback" tasks in an emergency.

Ansible Command with Tags:

- **--tags <tag_name> or -t <tag_name>:** Runs only tasks/plays with the specified tag(s).
 - **Example:** `ansible-playbook my_app.yml --tags deploy_web` (Only runs tasks tagged `deploy_web`).
- **--skip-tags <tag_name> or -s <tag_name>:** Skips tasks/plays with the specified tag(s).
 - **Example:** `ansible-playbook my_app.yml --skip-tags expensive_tests` (Skips tests tagged `expensive_tests`).

Multiple Tags: You can specify multiple tags separated by commas:

- `ansible-playbook my_app.yml --tags web,db` (Runs tasks tagged *either* `web` or `db`).

Example Playbook with Tags:

```
- name: Full Application Deployment
  hosts: all

  tasks:
    - name: Install base packages
      ansible.builtin.yum:
        name: "{{ item }}"
        state: present
      loop:
        - git
        - python3
      tags: [ 'packages', 'initial_setup' ]

    - name: Configure Web Server
      # ... Nginx config tasks ...
      tags: [ 'web', 'configure' ]

    - name: Deploy application code
      # ... code deployment tasks ...
      tags: [ 'deploy', 'web' ]

    - name: Configure Database
```

```
# ... DB config tasks ...
tags: [ 'db', 'configure' ]
```

- `ansible-playbook my_playbook.yml -t web`: Would run "Configure Web Server" and "Deploy application code".
- `ansible-playbook my_playbook.yml -s initial_setup`: Would skip "Install base packages".

27. Ansible Roles

Definition: Roles are the **primary way to organize and reuse Ansible content** (tasks, handlers, variables, templates, files, metadata) in a standardized directory structure. They promote modularity, reusability, and maintainability of playbooks.

Real-time Example: Instead of copying Nginx installation tasks into every playbook that needs Nginx, you create an "nginx" role. Any playbook can then simply call that role.

28. Ansible Galaxy

Definition: Ansible Galaxy is the **official hub for finding, sharing, and collaborating on Ansible roles**. It's a community repository where users can upload and download roles created by others.

Ansible Galaxy Command (ansible-galaxy):

- **ansible-galaxy init <role_name>**: Creates a basic role directory structure.
 - **Use:** To start developing a new role.
 - **Example:** `ansible-galaxy init nginx_role` (Creates a directory `nginx_role` with subdirectories like `tasks/`, `handlers/`, `vars/`, etc.)
- **ansible-galaxy install <role_name>**: Downloads a role from Ansible Galaxy or a Git repository.
 - **Use:** To use a community-contributed role or a role from your own Git repository.
 - **Example (from Galaxy):** `ansible-galaxy install geerlingguy.nginx`
 - **Example (from Git):** `ansible-galaxy install git+https://github.com/myuser/my_ansible_role.git,main`

- **ansible-galaxy search <keyword>**: Searches for roles on Ansible Galaxy.
 - **Use:** To discover available roles.
 - **Example:** ansible-galaxy search jenkins

29. Role Directory Structure and Its Uses

When you run `ansible-galaxy init <role_name>`, it creates this standard structure:

```
my_role/
├── defaults/           # Default variables for the role (lowest
precedence)
│   └── main.yml
├── handlers/          # Handlers for the role (tasks that run when
notified)
│   └── main.yml
├── meta/               # Role metadata (author, description,
dependencies)
│   └── main.yml
├── tasks/              # Main tasks for the role (most common)
│   └── main.yml
├── templates/          # Jinja2 templates (dynamic configuration
files)
├── files/              # Static files (copied as-is)
├── vars/               # Variables for the role (higher precedence
than defaults)
│   └── main.yml
├── tests/              # Playbooks for testing the role
│   ├── inventory
│   └── test.yml
└── README.md           # Documentation for the role
```

Uses:

- **tasks/main.yml**: This is where the core automation logic of the role resides.
- **handlers/main.yml**: Defines actions that should only run when explicitly triggered by a task (e.g., service restarts).

- **defaults/main.yml:** Provides default values for variables. These can be easily overridden by higher-precedence variables (e.g., in vars/, inventory, or CLI).
- **vars/main.yml:** Variables specific to the role that override defaults.
- **files/:** Stores static files that are copied directly to managed nodes (e.g., .deb, .rpm, or non-templated config files).
- **templates/:** Stores Jinja2 templates that are processed before being copied to managed nodes (e.g., dynamic .conf files).
- **meta/main.yml:** Contains metadata about the role (author, description, license) and critically, its **dependencies** on other roles.

30. Ansible Playbook with Roles

To use a role, you simply reference it in the roles section of a play.

Example Playbook (site.yml):

```
---
- name: Deploy Web Application Stack
  hosts: webserver
  become: yes
  vars:
    web_app_port: 8080 # This variable can be used inside the
    webserver and firewall roles

  roles:
    - common          # Role for common system configurations (e.g.,
    NTP, users)
    - webserver        # Role for installing and configuring
    Nginx/Apache
    - firewall         # Role for managing firewall rules
```

Ansible will execute the tasks within each role in the order specified.

31. role.yml with Multiple Roles (Correct Context)

The concept of "role.yml with multiple roles" typically refers to two main scenarios:

1. **Multiple Roles in a Single Playbook:** As shown above in section 30, a single play in a playbook can apply multiple roles sequentially.
2. **Role Dependencies (in meta/main.yml):** A role itself can declare dependencies on other roles within its meta/main.yml file. This means when you call Role A, Ansible automatically includes Role B if Role A depends on it.

Example (within my_app_role/meta/main.yml):

```
# my_app_role/meta/main.yml
dependencies:
  - role: common_server_setup
  - role: nginx_install
  vars:
    nginx_listen_port: 80
  - role: database_client_config
```

In this scenario, if you include my_app_role in your playbook, Ansible will first execute common_server_setup, then nginx_install (with the specified variable), then database_client_config, and finally the tasks within my_app_role itself. This is powerful for building complex, reusable components.

32. Ansible Tower

Definition: Ansible Tower (now branded as **Red Hat Ansible Automation Platform**) is a **commercial web-based UI and API solution** built on top of Ansible. It provides a centralized dashboard, role-based access control (RBAC), job scheduling, auditing, and graphical inventory management for large-scale Ansible deployments.

Real-time Use Cases:

- **Large Enterprises:** For managing hundreds or thousands of servers.
- **Teams:** Provides a GUI for non-CLI users to run playbooks, enforce permissions.
- **Compliance & Auditing:** Tracks who ran what, when, and with what results.
- **Integration:** Integrates with CMDB, ITSM, and CI/CD pipelines.

33. Ansible AWX in Detail

Definition: AWX is the **open-source upstream project that Ansible Tower (Red Hat Ansible Automation Platform) is based on**. It provides almost all the core features of Tower but is community-supported and requires self-hosting and maintenance.

Key Features (similar to Tower):

- **Web-based User Interface:** A dashboard to manage and launch Ansible playbooks.
- **Role-Based Access Control (RBAC):** Define permissions for users and teams to specific projects, inventories, and playbooks.
- **Inventory Management:** Manage static and dynamic inventories from the UI.
- **Job Scheduling:** Schedule playbooks to run at specific times or intervals.
- **Credential Management:** Securely store SSH keys, passwords, and other credentials using Vault or its internal credential manager.
- **Monitoring & Logging:** Centralized logging and real-time output of playbook runs.
- **API:** A RESTful API for programmatic interaction and integration with other systems.

Real-time Use Cases (for AWX):

- **Smaller Teams/Organizations:** Who need the benefits of Tower but prefer an open-source, self-hosted solution.
- **Learning & Development:** A great way to explore the features of Ansible Automation Platform without commercial licensing.
- **Customization:** Offers more flexibility for deep customization as it's open source.

Deployment: AWX is typically deployed using Kubernetes/OpenShift (or Docker Compose for development environments).

34. Missing Important Things (for 5-year experience level)

Here are a few more crucial concepts for a deeper understanding:

- **Idempotence:**
 - **Definition:** Ansible tasks are designed to be idempotent, meaning they can be run multiple times without causing unintended side effects or making changes if the target system is already in the desired state.

- **Real-time Example:** Running `apt: name=nginx state=present` multiple times will only install Nginx if it's missing. If it's already installed, the task will report "OK" (not "CHANGED"). This is fundamental to reliable automation.
 - **Facts (ansible_facts):**
 - **Definition:** System information (OS, memory, network interfaces, installed packages) gathered automatically by Ansible at the beginning of a play (unless `gather_facts: no`). These are stored in the `ansible_facts` variable.
 - **Real-time Example:** Using `{{ ansible_os_family }}` in a when condition or `{{ ansible_default_ipv4.address }}` in a template.
 - **Delegation (delegate_to, run_once):**
 - **Definition:** Allows a task that is part of a play targeting one group of hosts to be executed on a *different* host (the `delegate_to` host) or to run only once on the control node.
 - **Real-time Example:**
 - `delegate_to: localhost`: Useful for tasks that interact with local machine (e.g., triggering a Jenkins job or creating a local temporary file).
 - `run_once: true`: Used for tasks that should only run once, even if the play targets multiple hosts (e.g., creating a single entry in a database or cloud resource).
 - **Error Handling (ignore_errors, failed_when, changed_when):**
 - **Definition:** Mechanisms to control how Ansible reacts to task failures.
 - **ignore_errors: true:** Continues playbook execution even if a task fails.
 - **failed_when::** Defines custom conditions under which a task is considered failed (e.g., if a command's output contains "Error").
 - **changed_when::** Defines custom conditions under which a task is considered "changed" (not just based on module's return).
 - **Real-time Example:**
- name: Attempt to start service (ignore if it fails)
 ansible.builtin.service: name=my_app state=started
 ignore_errors: yes
- name: Check custom script output
 ansible.builtin.command: /opt/my_script.sh
 register: script_output
 failed_when: "'ERROR' in script_output.stderr" # Task fails if ERROR

is in stderr

- **Collections:**
 - **Definition:** The modern way to package and distribute Ansible content (modules, plugins, roles, playbooks). They provide a more modular and versioned approach to content management, separating core Ansible from community/vendor content.
 - **Real-time Example:** Most new modules are part of collections (e.g., `ansible.builtin.copy` is implicitly the `ansible.builtin` collection; `community.aws.ec2_instance` is from the `community.aws` collection). You install them via `ansible-galaxy collection install`.
- **check_mode (Dry Run):**
 - **Definition:** Running a playbook with `--check` or `--dry-run` flag. Ansible will report what *would* have changed without actually making any modifications on the managed nodes.
 - **Real-time Example:** Testing a new playbook in a production environment to see its impact before committing to changes.
- **diff_mode:**
 - **Definition:** Running a playbook with `--diff` flag. Ansible will show the differences in files and templates that it would have made.
 - **Real-time Example:** Before applying a configuration, reviewing the exact changes that will be pushed.